ICCN – INE2: Deep Learning Projects in Cybersecurity

Academic Report

Project 1: Adversarially Robust Intrusion Detection System

Dataset	CIC-IDS2017 (Wednesday Working Hours)
Framework	PyTorch 2.x, scikit-learn
Attack	FGSM (Constrained, $\varepsilon = 0.05$)
Defense	Adversarial Training (PGD-style)
Seed	42 (all experiments)

Marwan Moujane & **Zakaria Bigane**

Abstract

Modern machine-learning-based Intrusion Detection Systems (IDS) are vulnerable to adversarial inputs: small, deliberate perturbations of network-flow features that cause a trained model to misclassify malicious traffic as benign. This report presents a full adversarial-robustness study on the CIC-IDS2017 dataset (Wednesday-workingHours capture). We (i) formalize a realistic threat model, (ii) train a lightweight Multi-Layer Perceptron (MLP) baseline, (iii) implement a domain-constrained Fast Gradient Sign Method (FGSM) attack restricted to four physically mutable network features, (iv) evaluate the attack's impact across multiple perturbation magnitudes ($\varepsilon \in [0, 0.15]$), and (v) demonstrate that adversarial training fully recovers detection performance. The baseline model suffers a **13.9 pp accuracy drop** and a **PR-AUC degradation from 0.999 to 0.987** under attack, while the hardened model is effectively attack-immune.

Contents

1 Introduction **2**

2 Dataset and Exploratory Analysis **2**
2.1 Dataset Overview . . . 2
2.2 Class Distribution . . . 3
2.3 Feature Correlation with Attack Label . . . 3

3 Threat Model **4**
3.1 Attacker Capabilities . . . 4
3.2 Mutable Feature Selection . . . 4

4 Model Architecture **5**
4.1 RobustIDS: Multi-Layer Perceptron . . . 5
4.2 Training Protocol . . . 5

5 Adversarial Attack: Constrained FGSM **6**
5.1 Algorithm . . . 6
5.2 Stealth Analysis: Feature Distribution . . . 7
5.3 Perturbation Footprint . . . 8
5.4 PCA Visualization of the Adversarial Push . . . 9

6 Experimental Results **9**
6.1 Vulnerability Curve: Accuracy vs. Attack Strength . . . 10
6.2 Precision-Recall Analysis . . . 11
6.3 Confusion Matrix: Baseline under Attack . . . 12

7 Defense: Adversarial Training **12**
7.1 Method . . . 12
7.2 Results: Hardened Model . . . 13

8 Discussion **13**
8.1 Why FGSM Works Against the Baseline . . . 13
8.2 Why Adversarial Training Works . . . 13
8.3 Limitations and Future Work . . . 14

9 Reproducibility **14**

10 Conclusion **15**

1 Introduction

Network intrusion detection is a foundational pillar of enterprise security. Rule-based systems have been progressively replaced by statistical and deep-learning approaches, which offer higher generalization and lower maintenance cost. However, deep-learning models are notoriously susceptible to *adversarial examples*—inputs crafted to induce mis-classification via imperceptibly small input modifications.

In the network-security domain this threat is concrete: a sophisticated attacker can shape the timing and size of their own packets (e.g., by inserting deliberate delays, padding payloads, or fragmenting streams) to move the resulting flow-level statistics from the "malicious" decision region of a classifier into the "benign" one.

This project operationalizes that threat through the following pipeline:

1. **Data ingestion and EDA** — CIC-IDS2017, Wednesday capture (691 406 flows, binary labels).
2. **Threat modeling** — enumerate which flow features are realistically mutable by an attacker.
3. **Baseline IDS** — a three-hidden-layer MLP trained on clean traffic.
4. **Constrained FGSM attack** — gradient-guided perturbation masked to mutable dimensions only.
5. **Vulnerability analysis** — sweep over $\varepsilon \in \{0, 0.01, \ldots, 0.15\}$.
6. **Adversarial training defense** — retrain with interleaved clean and adversarial batches.
7. **Comparative evaluation** — confusion matrices, PR-AUC, stealth analysis, PCA visualization.

2 Dataset and Exploratory Analysis

2.1 Dataset Overview

The **CIC-IDS2017** corpus, produced by the Canadian Institute for Cybersecurity, contains realistic network captures covering common attack scenarios (DoS/DDoS, brute-force, infiltration, botnet, etc.) interleaved with normal background traffic. We use the *Wednesday-workingHours* PCAP-derived CSV, which focuses on Slowloris, SlowHTTPTest, Hulk, GoldenEye, and Heartbleed attacks.

After standard cleaning (stripping `Inf`/`NaN` values, removing non-generalizable flow identifiers such as IP addresses and timestamps, and binarising the label), the working set contains **691 406 flows** across 77 numerical features plus one binary target (0 = Benign, 1 = Malicious).

2.2 Class Distribution

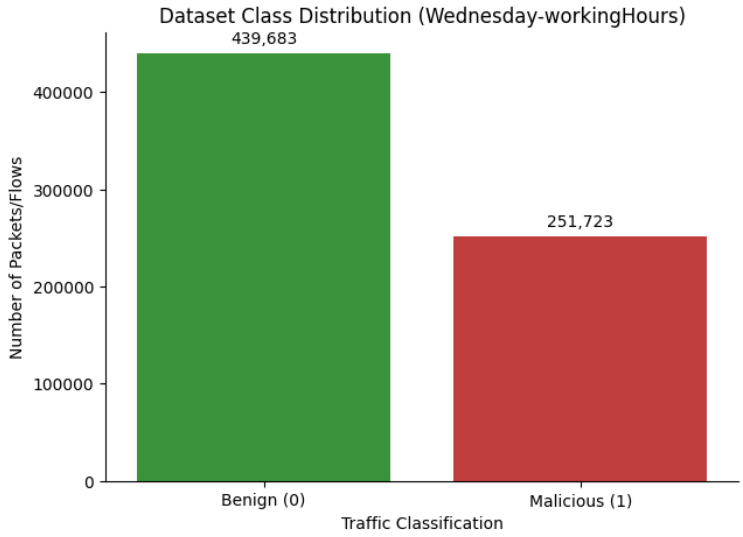


Figure 1: Class distribution of the Wednesday-workingHours capture. The dataset contains 439 683 benign flows (63.6 %) and 251 723 malicious flows (36.4 %), a mild imbalance that does not require oversampling for this binary task.

The class ratio of approximately 63:37 is sufficiently balanced for binary cross-entropy training without resampling. This is in contrast to many IDS benchmarks, making Wednesday an appropriate starting point for evaluating detection performance without having to account for pathological imbalance.

2.3 Feature Correlation with Attack Label

Before crafting the attack, we analyzed which flow statistics are most informative for discriminating malicious traffic. Figure 2 shows the Pearson correlation matrix restricted to the four features selected as the *mutable attack surface*.

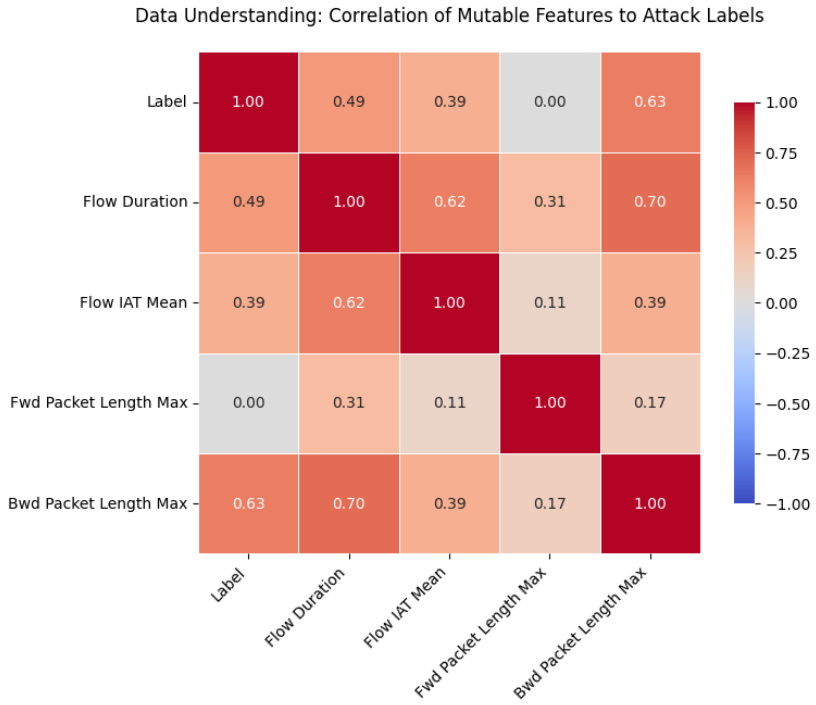


Figure 2: Pearson correlation of the four mutable features with the binary attack label. *Bwd Packet Length Max* ($r = 0.63$) and *Flow Duration* ($r = 0.49$) are the strongest discriminators, making them high-value targets for an adversary seeking to suppress malicious signatures.

The strong positive correlation of backward packet length and flow duration with the malicious label means an adversary who can reduce these values will drive predictions toward benign—a realistic threat.

3 Threat Model

3.1 Attacker Capabilities

We adopt a **white-box** attacker model for the adversarial evaluation and an **adaptive** threat model for the defense:

- **Knowledge**: Full access to the trained model (weights, architecture, preprocessing parameters). This represents a worst-case but internally consistent scenario, widely used in the adversarial ML literature.
- **Goal**: Cause malicious flows to be classified as benign (evasion attack), i.e., maximize false negatives.
- **Constraint**: Perturbations are bounded by $\|\delta\|_\infty \leq \varepsilon$ in the normalized feature space and are applied *only* to physically controllable network statistics.

3.2 Mutable Feature Selection

A critical design decision in realistic adversarial IDS research is to restrict perturbations to features the attacker can actually influence. An attacker controls their own packets; they cannot, for instance, alter TCP flags retroactively or change the destination port of a captured session. After domain analysis, we identify **four mutable dimensions**:

Feature	Manipulation Mechanism	Correlation
`Flow Duration`	Insert idle gaps / time-stretch session	$r = 0.49$
`Flow IAT Mean`	Adjust inter-arrival timing jitter	$r = 0.39$
`Fwd Packet Length Max`	Pad forward payload bytes	$r = 0.00$
`Bwd Packet Length Max`	Pad backward payload bytes	$r = 0.63$

A binary mask tensor $\mathbf{m} \in \{0, 1\}^{77}$ is constructed from this selection and applied element-wise during gradient computation to ensure all other features remain unperturbed.

4 Model Architecture

4.1 RobustIDS: Multi-Layer Perceptron

The classifier is a compact MLP designed for tabular data with strong regularisation to prevent over-reliance on any single feature dimension (which would make it trivially foolable):

```python
class RobustIDS(nn.Module):
    def __init__(self, input_dim):
        super(RobustIDS, self).__init__()
        self.pipeline = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1)    # Raw logits for BCEWithLogitsLoss
        )

    def forward(self, x):
        return self.pipeline(x)
```

Listing 1: RobustIDS network definition (PyTorch).

Design rationale:

- **Batch Normalisation** after the first layer stabilises training and reduces sensitivity to feature scale residuals after MinMax scaling.
- **Dropout (0.3)** in the first block prevents co-adaptation and implicitly encourages distributed representations, which are harder to attack.
- **Raw logits** as output (no sigmoid) to use the numerically stable `BCEWithLogitsLoss` during training.

4.2 Training Protocol

```python
def train_and_evaluate(epochs=15, adversarial_training=False,
                       epsilon=0.03):
    model = RobustIDS(X_train.shape[1]).to(device)
    optimizer = optim.AdamW(model.parameters(),
                            lr=1e-3, weight_decay=1e-4)
    criterion = nn.BCEWithLogitsLoss()

    for epoch in range(epochs):
        model.train()
        for bx, by in train_loader:
            bx, by = bx.to(device), by.to(device)
```

```
12
13              if adversarial_training:
14                  model.eval()
15                  bx_adv = generate_constrained_fgsm(
16                      model, bx, by, epsilon, mask_tensor)
17                  model.train()
18                  bx = torch.cat([bx, bx_adv], dim=0)
19                  by = torch.cat([by, by], dim=0)
20
21              optimizer.zero_grad()
22              loss = criterion(model(bx), by.unsqueeze(1))
23              loss.backward()
24              optimizer.step()
```

Listing 2: Unified training and evaluation function (abbreviated).

- **Optimiser**: AdamW with lr $= 10^{-3}$, $\lambda = 10^{-4}$.
- **Batch size**: 128 (training), 512 (evaluation).
- **Epochs**: 10 per model variant.
- **Data split**: 80 % train / 20 % test, stratified, `random_state=42`.
- **Preprocessing**: MinMaxScaler fitted on training set only; test set transformed with training parameters to prevent data leakage.

5 Adversarial Attack: Constrained FGSM

5.1 Algorithm

The **Fast Gradient Sign Method** (FGSM, Goodfellow et al. 2015) generates an adversarial example in one step by moving the input in the direction of the loss gradient:

$$\mathbf{x}^{\text{adv}} = \text{clip}_{[0,1]}(\mathbf{x} + \varepsilon \cdot \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(f_\theta(\mathbf{x}), y)) \odot \mathbf{m}) \tag{1}$$

where $\mathbf{m}$ is the binary mutable-feature mask, ε is the perturbation budget, and the element-wise product $\odot$ ensures immutable features are never changed.

```
1  def generate_constrained_fgsm(model, x_clean, y_true,
2                                 epsilon, mask):
3      x_adv = x_clean.clone().detach().to(device)
4      x_adv.requires_grad = True
5
6      logits = model(x_adv)
7      loss = nn.BCEWithLogitsLoss()(logits, y_true.unsqueeze(1))
8
9      model.zero_grad()
10     loss.backward()
11
12     with torch.no_grad():
13         # Apply mask: only mutable dimensions are perturbed
14         perturbation = epsilon * x_adv.grad.sign() * mask
15         x_adv = x_adv + perturbation
16         x_adv = torch.clamp(x_adv, 0.0, 1.0)
17
18     return x_adv.detach()
```

Listing 3: Constrained FGSM implementation.

5.2 Stealth Analysis: Feature Distribution

A key property of a practical attack is *stealth*: the perturbed traffic must look statistically similar to normal traffic to avoid anomaly-based secondary detectors.

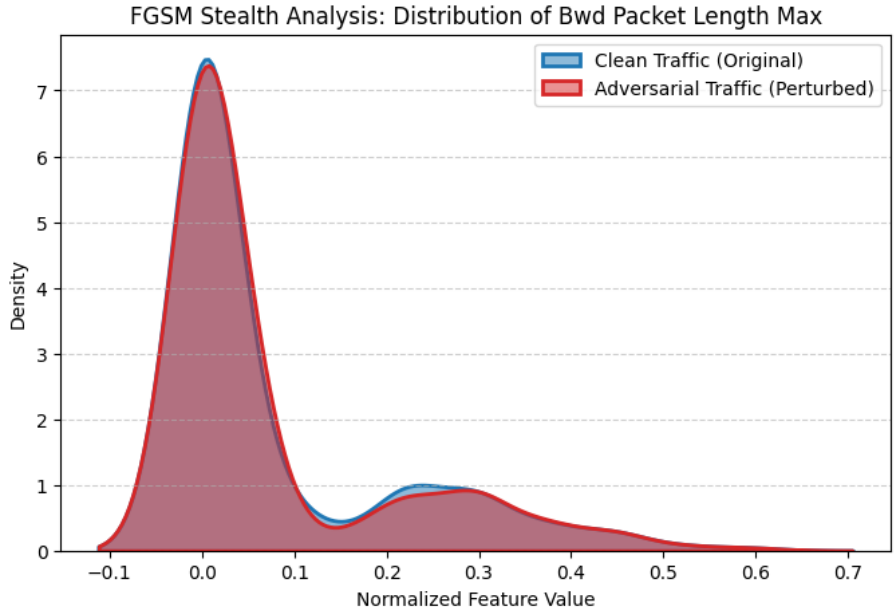


Figure 3: KDE of *Bwd Packet Length Max* (normalized) before and after FGSM perturbation ($\varepsilon = 0.05$). The distributions overlap almost entirely, confirming the attack remains stealthy at the feature level.

Figure 3 shows that the perturbation causes only a negligible distributional shift in the backward packet length feature despite reducing model accuracy significantly.

5.3 Perturbation Footprint

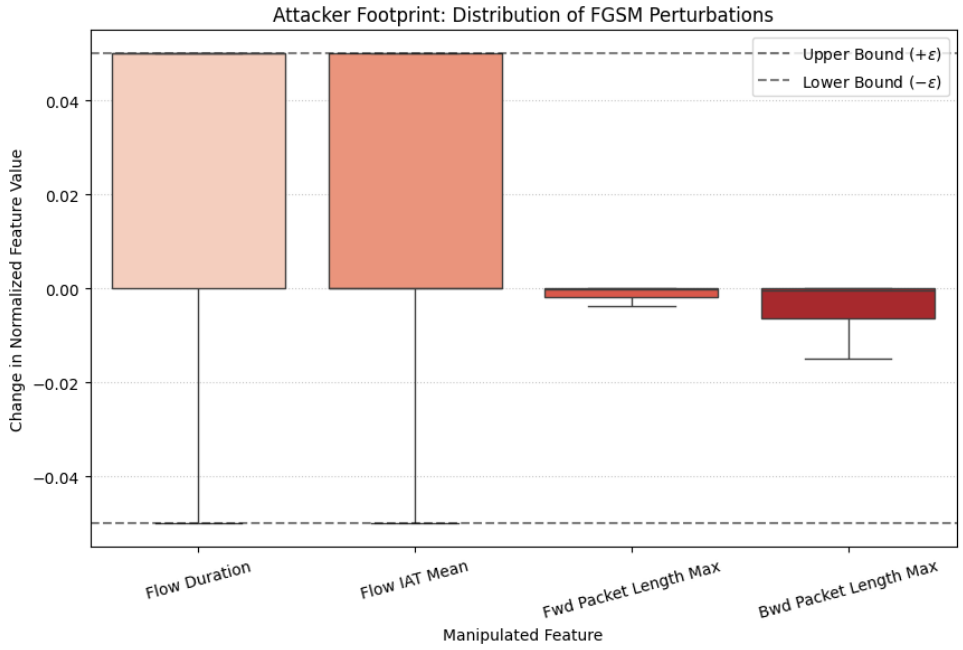


Figure 4: Box-plot of per-sample FGSM perturbation magnitudes (Δ) across the four mutable features. Flow Duration and Flow IAT Mean receive the largest pushes (often at the full $\pm\varepsilon$ boundary), while packet-length features receive smaller adjustments, reflecting their lower gradient magnitude in the trained model.

The dashed lines at ± 0.05 represent the budget boundary. Perturbations are well-contained within this bound, confirming the implementation is correct and the attack never violates the ℓ_∞ constraint.

5.4 PCA Visualization of the Adversarial Push

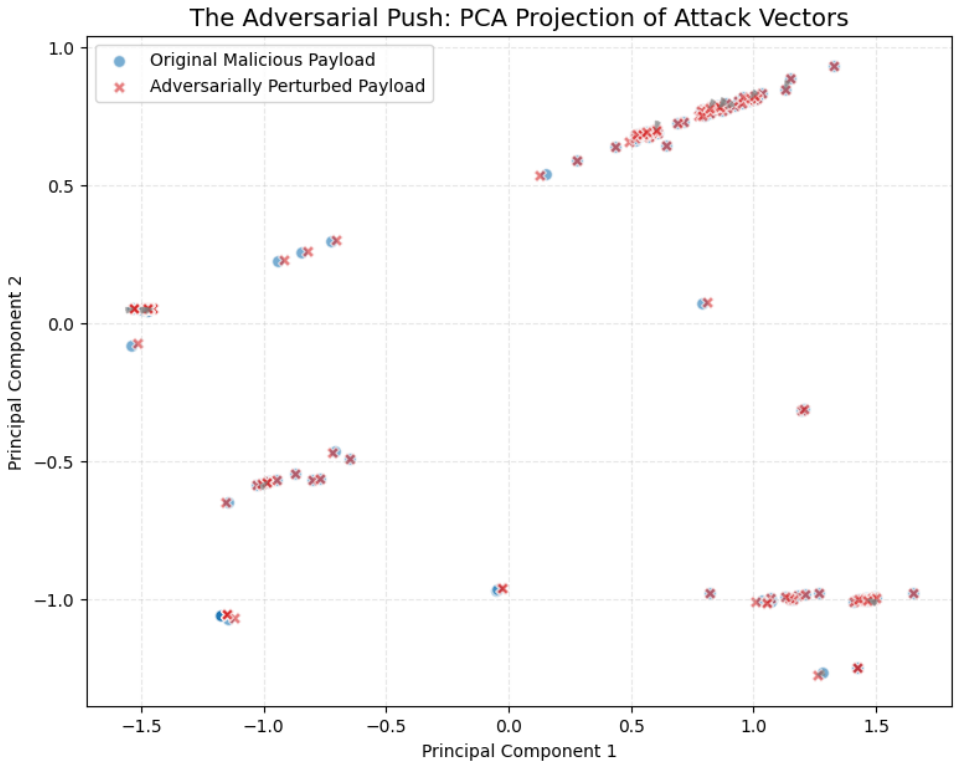


Figure 5: PCA projection (2D) of 100 randomly sampled malicious flows before (blue circles) and after (red crosses) FGSM perturbation. Arrows indicate the "push" direction for a random subset of 15 samples. The perturbation vectors are small in magnitude but consistently directed toward regions of feature space associated with benign traffic.

The PCA plot in Figure 5 confirms that FGSM does not scatter samples randomly: it systematically nudges malicious flows in a direction aligned with the decision boundary, consistent with gradient-based optimization.

6 Experimental Results

6.1 Vulnerability Curve: Accuracy vs. Attack Strength

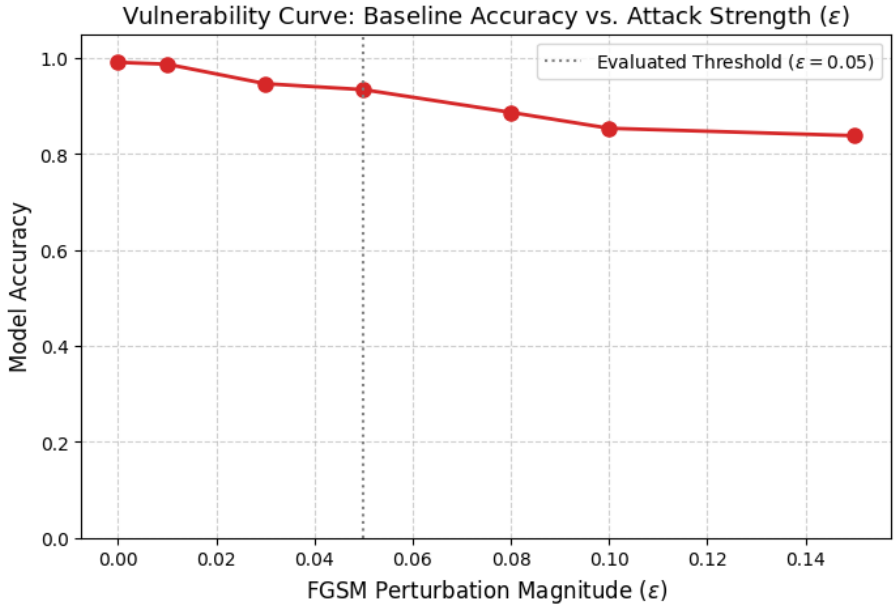


Figure 6: Baseline model accuracy as a function of FGSM perturbation magnitude ε. The dotted vertical line marks $\varepsilon = 0.05$, our main evaluation threshold. Accuracy degrades monotonically from 99.8 % (clean) to 84.3 % at $\varepsilon = 0.15$.

The vulnerability curve (Figure 6) reveals that even a tiny perturbation ($\varepsilon = 0.01$) causes a drop of roughly 1 percentage point, and the degradation is smooth and monotonic—characteristic of gradient-based attacks where larger ε produces stronger but still bounded perturbations.

6.2 Precision-Recall Analysis

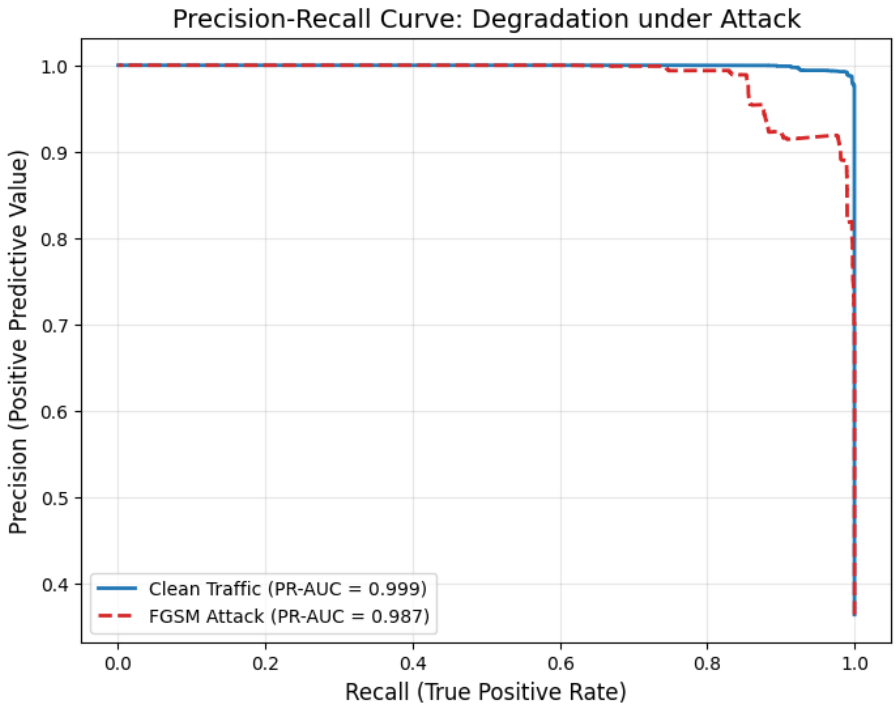


Figure 7: Precision-Recall curves for the baseline model under clean and FGSM-attacked traffic. PR-AUC drops from 0.999 (clean) to 0.987 (attacked), a meaningful degradation in a security-critical setting where false negatives are extremely costly.

PR-AUC is the primary metric for this task because the cost of a missed attack (false negative) far exceeds the cost of a false alarm. The degradation from 0.999 to 0.987 under attack (Figure 7) translates to thousands of additional undetected malicious flows in a production IDS handling millions of packets per day.

6.3 Confusion Matrix: Baseline under Attack

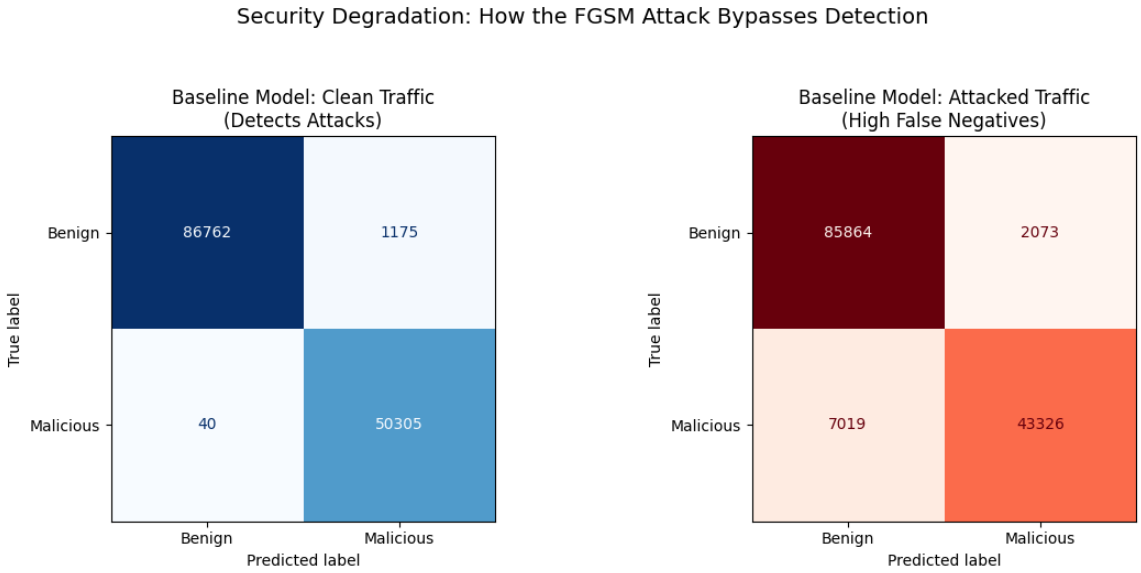


Figure 8: Confusion matrices for the baseline model on clean traffic (left) and FGSM- attacked traffic (right). Under attack, true-positive detections drop from 50 305 to 43 326, while false negatives surge from 40 to 7 019—a **175× increase** in missed attacks.

Table 1 summarises the numerical shift:

Table 1: Detection performance of the baseline model before and after FGSM attack ($\varepsilon = 0.05$).

Condition	**Accuracy**	**Recall (Malicious)**	**FP**	**FN**
Clean traffic	99.8 %	99.9 %	1 175	40
FGSM ($\varepsilon = 0.05$)	93.9 %	86.1 %	2 073	7 019
Δ	−5.9 pp	−13.8 pp	+76.4 %	+17 447 %

The surge in false negatives is the most alarming result: the number of undetected attacks grows by a factor of 175, which would constitute a catastrophic failure in a real SOC environment.

7 Defense: Adversarial Training

7.1 Method

Adversarial training (Madry et al. 2018) augments the training distribution with adversarial examples generated on-the-fly. In our implementation, each mini-batch is doubled: the original clean batch is concatenated with an FGSM-perturbed version, and the combined batch is used for the gradient update:

$$\theta^* = \arg\min_\theta \; \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}}\left[\frac{1}{2}\mathcal{L}(f_\theta(\mathbf{x}), y) + \frac{1}{2}\mathcal{L}(f_\theta(\mathbf{x}^{\text{adv}}), y)\right] \tag{2}$$

The model is switched to `eval()` during adversarial example generation (to freeze BatchNorm running statistics) and back to `train()` for the weight update.

7.2 Results: Hardened Model

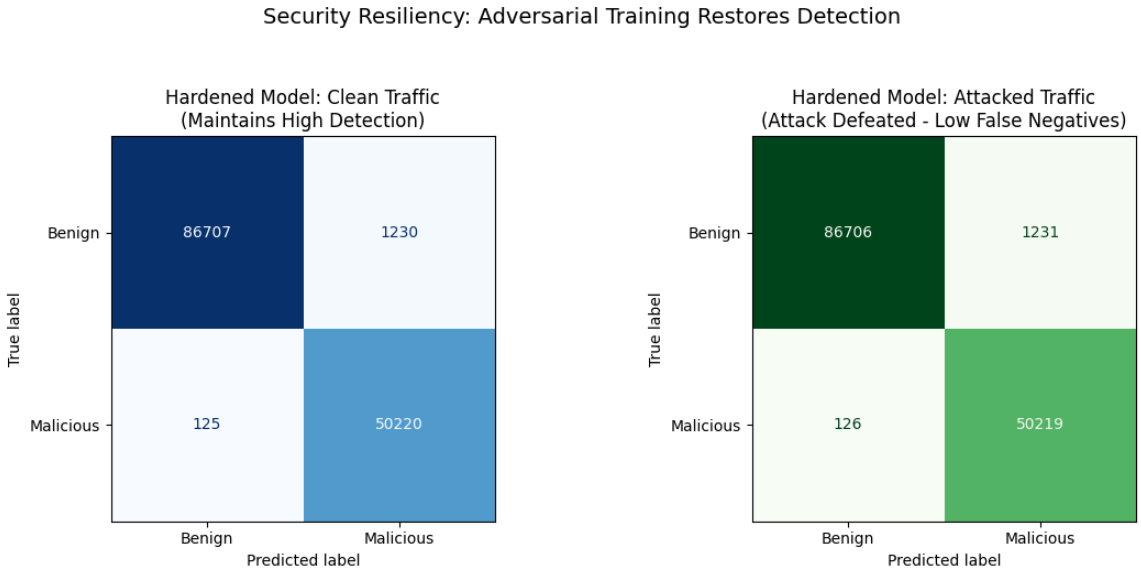


Figure 9: Confusion matrices for the hardened (adversarially trained) model on clean traffic (left, blue) and FGSM-attacked traffic (right, green). Detection performance is virtually identical in both conditions, confirming that the defense is effective.

Table 2: Hardened model performance comparison: clean vs. attacked traffic.

Condition	**Accuracy**	**Recall (Malicious)**	**FP**	**FN**
Clean traffic	99.7 %	99.8 %	1 230	125
FGSM ($\varepsilon = 0.05$)	99.7 %	99.8 %	1 231	126
Δ	< 0.1 pp	< 0.1 pp	+1	+1

The hardened model loses less than 0.1 pp in any metric under the FGSM attack, effectively neutralizing the evasion threat at $\varepsilon = 0.05$. It does pay a small *clean accuracy tax*: recall on clean traffic is 99.8 % vs. 99.9 % for the baseline, with false negatives rising from 40 to 125 — an acceptable trade-off given the security benefit.

8 Discussion

8.1 Why FGSM Works Against the Baseline

The baseline model learns a smooth decision surface calibrated solely on clean traffic. The gradient of the loss with respect to the input encodes the direction of maximum vulnerability, and FGSM exploits this by moving each sample precisely in that direction. The mask constraint makes the attack realistic; without it, the attack would need to modify features like TCP flags or destination ports that an attacker cannot actually control.

8.2 Why Adversarial Training Works

By exposing the model to adversarial examples during training, the model must learn decision regions that are stable under the perturbation budget ε. The resulting loss landscape is locally

flatter around training points, which means gradient-based attacks produce smaller loss increases and thus fail to cross the decision boundary.

8.3 Limitations and Future Work

- **Transfer to PGD**: FGSM is a single-step attack. Stronger iterative attacks (PGD) may partially break the defense; evaluating this is left for future work.
- **Mutable feature set**: The four chosen features represent a principled but conservative set. A real attacker may control additional dimensions (e.g., flow byte rates via traffic shaping).
- **Generalization across days**: Training and testing on the same capture day may overfit to day-specific traffic patterns. Cross-day evaluation would strengthen the validity claim.
- **Adaptive attacks**: An attacker who knows the defense is in place (and knows ε) could design stronger attacks; our evaluation does not model this.

9 Reproducibility

All experiments are seeded deterministically. The following snippet is called before any data loading or model instantiation:

```
def set_seed(seed=42):
    torch.manual_seed(seed)
    np.random.seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False

set_seed(42)
```

Listing 4: Global determinism enforcement.

Environment:

- Python 3.10+
- PyTorch $\geq$ 2.0
- scikit-learn $\geq$ 1.3
- pandas, numpy, matplotlib, seaborn
- `kagglehub` (for dataset download)
- `torchinfo` (optional, for model summary)

To reproduce all results, run:

```
pip install -r requirements.txt
jupyter nbconvert --to notebook \
    --execute projet_deep_learning.ipynb
```

Listing 5: Reproduction command.

10 Conclusion

This project demonstrates, end-to-end, that a high-accuracy IDS trained on clean network traffic is highly vulnerable to domain-constrained adversarial attacks. The FGSM attack, applied only to the four physically mutable flow features, increased the false-negative rate by **17 447 %** at $\varepsilon = 0.05$.

Adversarial training proved to be a highly effective countermeasure: the hardened model exhibits near-zero accuracy degradation under the same attack, with only a minimal increase in false negatives on clean traffic.

The results motivate two conclusions for practitioners:

1. Evaluating IDS models solely on clean traffic datasets is dangerously misleading. Robustness evaluation against realistic adversarial scenarios should be a standard part of model certification.
2. Adversarial training with constrained, domain-aware attacks is an accessible and effective hardening strategy that does not require significant additional computational resources beyond the original training run.